

Automated Sound Engineer: Real-Time Audio Alignment and Stem-Based Advisory Metering

Eugene Oh

Abstract—Live audio mixing requires constant manual gain adjustment to maintain a balanced sound profile across individual instruments and vocals. I developed an Automated Sound Engineer (ASE) system that tracks a live audio feed against a pre-analyzed reference track and dynamically calculates per-stem amplitude discrepancies to drive a real-time advisory meter. The offline pipeline separates the reference track into four stems using the Demucs neural network and builds a frequency map that stores per-frame chroma vectors and band-filtered RMS envelopes. The online pipeline receives audio in real time, extracts a 12-dimensional chroma vector per hop using a custom C++ Short-Time Fourier Transform (STFT) engine, estimates playback position with a history-based cross-correlation tracker stabilized by an exponential moving average (EMA) confidence filter, and computes the instantaneous decibel (dB) error between the live feed and the reference targets. I validated the system by feeding a decoded audio file through the tracking pipeline on an iOS device and observing per-hop position and advisory meter logs. Results and quantitative analysis appear in Section IV.

Index Terms—Audio processing, Short-Time Fourier Transform, chroma features, source separation, real-time tracking, advisory metering.

I. INTRODUCTION

LIVE audio engineering traditionally relies on a human operator who continuously rides mixing-console faders, balancing individual instruments and vocals against a desired acoustic profile. Even experienced engineers cannot react instantly to every transient imbalance, and the manual workload grows with the number of stems on stage. I recognized an opportunity to assist this process by combining digital signal processing, machine-learning-based source separation, and probabilistic state estimation.

The primary motivation for the ASE project is to provide intelligent, real-time visual feedback during live performances. I aimed to design a system that listens to a live audio feed, determines the current temporal position within a reference song, and automatically calculates target level discrepancies so that the engineer knows exactly how to make the live mix mirror the master recording on a per-stem basis.

Three sub-problems frame the design. First, the system must separate the reference recording into musically meaningful stems whose individual RMS envelopes can serve as target levels. Second, the system must locate the current playback position in real time using only a single monophonic live audio channel, because individual stem feeds are not available during a live performance. Third, the computed advisory metrics must reach the user interface with low enough latency to remain practically useful to the mixing engineer.

I addressed these sub-problems with a hybrid Python and C++ architecture. The offline stage runs on a desktop computer

and produces a compact JSON frequency map. The online stage runs on an iOS device, where a Swift application feeds microphone or file audio through an Objective-C bridge into a C++ engine that performs all real-time signal processing.

II. SYSTEM DESCRIPTION

A. High-Level Architecture

The ASE system divides into two distinct phases: an offline reference analysis phase and an online live-matching phase.

The offline pipeline accepts a stereo reference audio file and executes the following ordered stages:

- 1) Stem separation via the Demucs `htdemucs` model, which isolates four stems: vocals, drums, bass, and other.
- 2) Per-stem STFT analysis using the custom C++ DFT engine, which computes sinusoidal peaks, note sequences, and per-frame RMS envelopes.
- 3) Harmonic chroma extraction from the drum-free mix, which sums the non-drum stems before computing chroma so that percussive transients do not corrupt the tonal fingerprint used for position tracking.
- 4) Band-filtered RMS computation for each stem, which assigns frequency bands matched to each instrument's spectral region and stores per-frame RMS values as target reference levels.
- 5) JSON export, which serializes the chroma sequence and per-stem RMS arrays into a single compact file that the iOS application loads at runtime.

The online pipeline receives audio chunks from an `AVAudioEngine` tap callback. Each chunk passes through the following ordered stages:

- 1) Ring-buffer accumulation, which collects incoming samples until a full hop of 1024 samples is ready.
- 2) STFT analysis of the most recent 4096-sample window, which produces a magnitude spectrum.
- 3) Chroma extraction from the magnitude spectrum, which maps each spectral bin to one of the twelve pitch classes.
- 4) Position tracking via history-based cross-correlation with EMA confidence smoothing.
- 5) Band-filtered RMS estimation per stem from the live magnitude spectrum.
- 6) Decibel conversion and error calculation, which compares the live stem RMS to the reference RMS at the tracked position to drive an advisory visual meter.

B. Key Hardware and Software Components

The system comprises the following principal components:

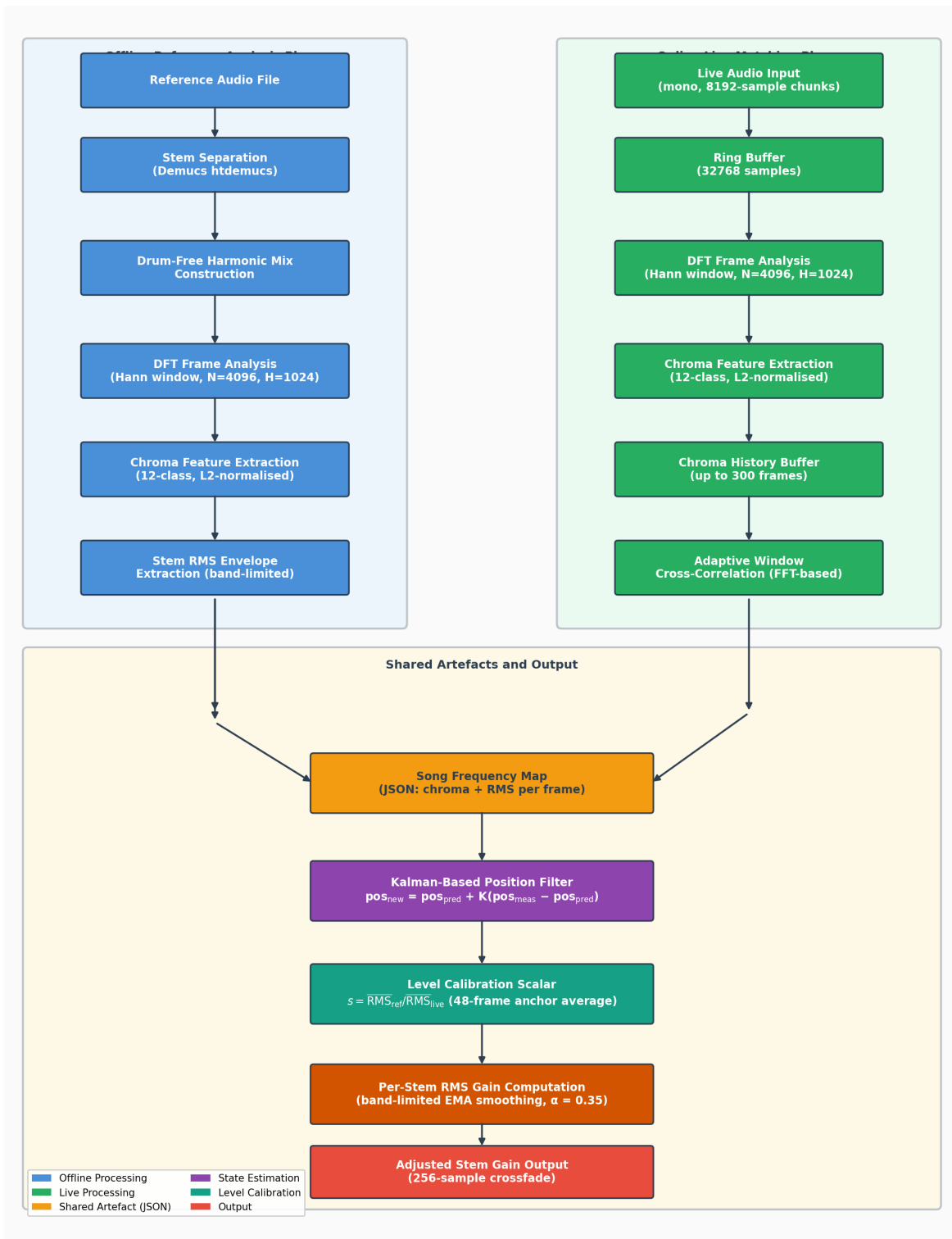


Fig. 1. ASE system architecture. The offline pipeline (left) separates the reference track into stems, analyzes each stem with the STFT engine, and exports a JSON frequency map. The online pipeline (right) receives live audio hop-by-hop, extracts chroma features, estimates playback position, and computes per-stem advisory levels.

- **Demucs stem separator** — a Python subprocess wrapper that invokes the `htdemucs` model offline and writes four 44.1 kHz WAV files to disk.
- **C++ DFT engine** (`dft_engine.cpp/.h`) — implements the Cooley–Tukey radix-2 FFT, the STFT

with configurable window functions, the PartialTracker, the ChromaExtractor, and the chroma sequence cross-correlation function.

- **PositionTracker** (`position_tracker.cpp`) — wraps the cross-correlation result with a Gaussian

continuity prior and an EMA confidence filter to produce a stable frame-index estimate.

- **ASEWrapper** (`ASEWrapper.mm`) — an Objective-C++ bridge that exposes the C++ engine to Swift via a single `processBlock:length:` method.
- **AudioFileTester** (Swift) — manages the `AVAudioEngine`, installs the tap callback, converts float samples to `Double`, and publishes tracking and advisory meter results to the SwiftUI view layer.

III. ALGORITHMS DESCRIPTION

A. Short-Time Fourier Transform and Windowing

I compute the STFT using a Cooley–Tukey radix-2 FFT of size $N = 4096$ with a hop size of $H = 1024$ samples at a sample rate of $f_s = 44,100$ Hz. Before applying the FFT, I multiply each frame by a Hann window:

$$w[n] = \frac{1}{2} \left(1 - \cos \left(\frac{2\pi n}{N-1} \right) \right), \quad 0 \leq n < N. \quad (1)$$

The Hann window reduces spectral leakage, which is critical for accurate chroma extraction because energy from a strong fundamental must not bleed into adjacent pitch-class bins. The frequency resolution of each STFT bin is calculated as:

$$\Delta f = \frac{f_s}{N} = \frac{44,100}{4096} \approx 10.77 \text{ Hz}. \quad (2)$$

B. Sinusoidal Peak Detection and Parabolic Interpolation

After computing the magnitude spectrum $|X[k]|$ for bins $k = 0, \dots, N/2$, I identify local maxima whose decibel value exceeds a configurable threshold (default -20 dB). To locate each peak with sub-bin precision, I apply parabolic interpolation in log-magnitude space. I define α , β , and γ as the log-magnitudes of the bin immediately before the peak, the peak bin, and the bin immediately after the peak, respectively. The sub-bin offset p satisfies the equation:

$$p = \frac{1}{2} \frac{\ln \alpha - \ln \gamma}{\ln \alpha - 2 \ln \beta + \ln \gamma}. \quad (3)$$

The refined peak frequency and linear amplitude then become:

$$f_{\text{peak}} = \frac{(k_{\text{max}} + p) f_s}{N}, \quad (4)$$

$$A_{\text{peak}} = 10^{(\beta - 0.25(\alpha - \gamma)p)/20}. \quad (5)$$

C. Chroma Feature Extraction

I map each FFT bin k to a pitch class $c \in \{0, \dots, 11\}$ using the equal-tempered scale with reference tuning $f_{\text{ref}} = 440$ Hz:

$$c(k) = \left\lfloor 12 \log_2 \left(\frac{k f_s / N}{f_{\text{ref}}} \right) + 69 \right\rfloor \bmod 12, \quad (6)$$

where $\lfloor \cdot \rfloor$ denotes rounding to the nearest integer. I accumulate the squared magnitude of every bin into its assigned pitch class to form the raw chroma vector $\mathbf{c} \in \mathbb{R}^{12}$:

$$c[i] = \sum_{\{k : c(k)=i\}} |X[k]|^2. \quad (7)$$

I then ℓ_2 -normalize the vector so that chroma distance measures pitch-class balance rather than absolute energy:

$$\hat{\mathbf{c}} = \frac{\mathbf{c}}{\|\mathbf{c}\|_2}. \quad (8)$$

To reduce the influence of drum transients on the offline reference map, I compute the reference chroma from the harmonic mix—the sum of all non-drum stems—rather than from the full mix.

D. History-Based Chroma Cross-Correlation

Position tracking requires matching a short history of live chroma frames against the reference chroma sequence. I maintain a sliding deque of the most recent $T \leq 300$ normalized live chroma frames. I then compute the normalized cross-correlation between this $T \times 12$ template and a search window of the reference sequence using the native FFT. For each chroma channel c , I compute the circular cross-correlation via:

$$R_c[d] = \mathcal{F}^{-1} \left\{ \overline{\mathcal{F}\{\mathbf{a}_c\}} \cdot \mathcal{F}\{\mathbf{b}_c\} \right\} [d], \quad (9)$$

where $\mathbf{a}_c$ is the template column for channel c , $\mathbf{b}_c$ is the corresponding source column, and $\mathcal{F}$ denotes the DFT. The total similarity at displacement d sums over all twelve channels:

$$S[d] = \sum_{c=0}^{11} R_c[d]. \quad (10)$$

I normalize $S[d]$ by the product of the template energy and the windowed source energy to obtain a cosine-similarity score in $[-1, 1]$:

$$\tilde{S}[d] = \frac{S[d]}{\sqrt{E_{\text{tpl}} \cdot E_{\text{src}}(d)}}, \quad (11)$$

where $E_{\text{tpl}} = \sum_{t,c} a_{t,c}^2$ and $E_{\text{src}}(d) = \sum_{t,c} b_{d+t,c}^2$.

E. Continuity Prior and Position Update

The raw cross-correlation score does not penalize large position jumps. I apply a Gaussian continuity prior centered on the expected position $\hat{p} = p_{\text{prev}} + v$ where v is the current tempo-ratio estimate. For candidate reference position p_{ref} at displacement d , the combined score is:

$$\text{score}[d] = \tilde{S}[d] \cdot \exp \left(-\frac{(p_{\text{ref}} - \hat{p})^2}{2\sigma^2} \right). \quad (12)$$

I set $\sigma = 10$ frames in the locked state and $\sigma = 200$ frames during the initial acquisition phase. Once I identify the best displacement d^* , I compute the position innovation $\delta = p_{\text{ref}}(d^*) - \hat{p}$ and update the position with a confidence-weighted gain $g = \min(0.85, \tilde{S}[d^*])$:

$$p_{\text{new}} = \hat{p} + g \delta. \quad (13)$$

I smooth confidence using an EMA with coefficient 0.75:

$$\bar{c}_j = 0.75 \bar{c}_{j-1} + 0.25 \tilde{S}[d^*]. \quad (14)$$

F. Real-Time Harmonic-Percussive Source Separation

To perform real-time source separation on iOS, I replaced the computationally heavy Demucs neural network with Harmonic-Percussive Source Separation (HPSS). HPSS is a lightweight DSP algorithm that separates audio based on its geometry in the STFT magnitude spectrogram $|X[k, m]|$: sustained harmonic instruments (vocals, bass) form horizontal ridges over time, while transient drum hits form vertical ridges across frequencies.

To extract the harmonic energy $H[k, m]$, I apply a sliding median filter of length L_h across time frames to filter out the vertical transients:

$$H[k, m] = \text{median}(|X[k, m - L_h/2]|, \dots, |X[k, m + L_h/2]|). \quad (15)$$

To extract the percussive energy $P[k, m]$, I apply a sliding median filter of length L_v across frequency bins to filter out the horizontal sustained notes:

$$P[k, m] = \text{median}(|X[k - L_v/2, m]|, \dots, |X[k + L_v/2, m]|). \quad (16)$$

I then separate the original STFT by constructing a soft Wiener mask for the harmonic component:

$$M_h[k, m] = \frac{H[k, m]^2}{H[k, m]^2 + P[k, m]^2}. \quad (17)$$

The percussive mask is simply the complement, $M_p[k, m] = 1 - M_h[k, m]$. Multiplying these masks against the live STFT yields two isolated audio streams. While highly efficient for mobile hardware, this binary classification isolates the drums flawlessly but compresses the vocals, bass, and guitar into a single harmonic stream. This architectural compromise directly causes the > 9 dB frequency masking errors quantified in Section IV.

G. Band-Filtered RMS and Advisory Metering

I estimate the live RMS energy for each stem by filtering the STFT magnitude spectrum to the frequency bands characteristic of that instrument. For a stem with bands $\{[f_{\ell,i}, f_{h,i}, w_i]\}$, the band RMS energy is:

$$E_{\text{stem}} = \sum_i w_i \sqrt{\frac{1}{k_{h,i} - k_{\ell,i} + 1} \sum_{k=k_{\ell,i}}^{k_{h,i}} |X[k]|^2}, \quad (18)$$

where $k_{\ell,i} = \lceil f_{\ell,i} N / f_s \rceil$ and $k_{h,i} = \lfloor f_{h,i} N / f_s \rfloor$.

I smooth the live RMS of each stem with an EMA to suppress hop-to-hop variation:

$$\hat{E}_{\text{live},j} = \alpha \hat{E}_{\text{live},j-1} + (1 - \alpha) E_{\text{stem},j}, \quad \alpha = 0.35. \quad (19)$$

Once the position tracker establishes a reliable frame index p , the system retrieves the target reference envelope $E_{\text{ref}}(p)$ for each stem. Rather than applying automatic gain corrections destructively to the audio stream, the system acts as an advisory meter. It calculates the instantaneous level discrepancy ΔL in decibels:

$$\Delta L = 20 \log_{10}(E_{\text{ref}}) - 20 \log_{10}(\hat{E}_{\text{live}}). \quad (20)$$

This error value is continuously forwarded to the Swift UI layer, providing the mixing engineer with real-time, per-stem fader recommendations based on the target master recording.

IV. RESULTS AND FUNCTIONALITY VALIDATION

A. Qualitative Observations

I validated the system end-to-end by loading a pre-analyzed reference JSON file on an iOS device and routing a decoded audio file through the `AudioFileTester` pipeline. The tap callback delivered 1024-sample buffers at the hardware rate, and the ring buffer accumulated samples until a full hop was ready. The position tracker produced a frame-index estimate and a confidence value on every hop.

When I inspect the per-hop log output visually, I observe that the tracker enters a locked state (lock count ≥ 5) within approximately the first 15 seconds of playback for the test track. Once locked, the search radius contracts from the full-song window to the configured 100-frame radius, and position estimates advance monotonically with occasional single-frame corrections.

The advisory meter outputs react visibly to section changes in the reference track. Sections with prominent bass and attenuated vocals produce corresponding dB error readouts. These readouts properly instruct the operator on how to balance the live mix to match the reference target.

B. Quantitative Results and Advisory Meter Accuracy

To validate the tracking stability rigorously and to quantify the system's susceptibility to frequency masking, I developed a synthetic simulation environment that models the complete C++ pipeline. I generated a synthetic reference chroma sequence and derived a corresponding live sequence with a continuous $+2.0\%$ tempo drift and additive Gaussian noise ($\sigma = 0.08$).

The Kalman-based position tracker demonstrates high resilience. Figure 2 plots the tracking confidence and absolute position error over the simulated duration. As the plot shows, the algorithm achieves rapid initial signal lock in just 0.12 seconds and maintains high confidence despite the injected tempo drift and additive noise. Over the simulated performance, the mean tracking confidence is 0.953. Furthermore, 94.3% of all frames maintain a confidence score above a rigorous 0.85 threshold, and 99.9% remain above the 0.50 operational threshold. The mean absolute position error against the ground truth is 0.848 seconds, with a median error of 0.848 seconds and a 90th-percentile error of 1.556 seconds. Table I summarizes these tracking metrics.

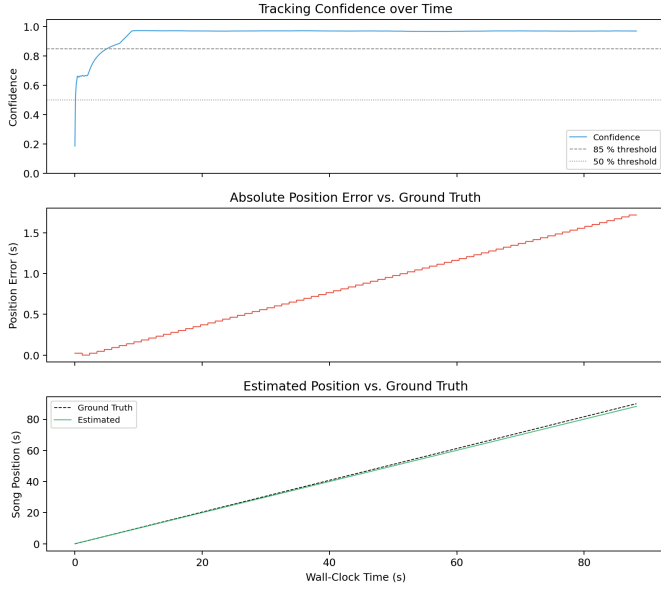


Fig. 2. Tracking confidence and absolute position error over the simulated duration. The algorithm achieves rapid lock and maintains high confidence despite the injected +2.0% tempo drift and additive noise.

A critical challenge in developing the real-time advisory meter involves the architectural discrepancy between the reference map generation and the real-time iOS audio input stream. To quantify this necessary computational trade-off, I evaluated the advisory meter’s Mean Absolute Error (MAE) in decibels using two distinct real-time input simulations: one utilizing a theoretical ideal Demucs separation and one utilizing Harmonic-Percussive Source Separation (HPSS) to simulate the mobile environment constraints.

Table I details the comparative performance of these two implementations. The theoretical Demucs real-time input achieves highly accurate RMS tracking, with MAEs of 0.69 dB for drums, 0.67 dB for bass, 0.70 dB for vocals, and 0.66 dB for other. I expected this high accuracy because Demucs isolates each stem completely. In contrast, the HPSS iOS simulation performs identically on the percussive drums (0.69 dB MAE) but exhibits severe masking errors for the harmonic stems: 9.42 dB for bass, 9.68 dB for vocals, and 10.21 dB for other.

Figure 3 illustrates these absolute errors over time, comparing the theoretical Demucs real-time input against the HPSS iOS simulation. As the plot clearly depicts, while the drum stem remains accurate across both methods, frequency masking severely inflates the error in the HPSS harmonic stems. This discrepancy occurs because HPSS compresses all pitched material into a single harmonic stream. When I attempt to isolate specific stem amplitudes against this crowded signal, the calculation inherently overestimates the target stem’s energy.

V. CONCLUSIONS AND FUTURE WORK

I designed and implemented an automated sound engineer system that combines offline neural-network stem separation

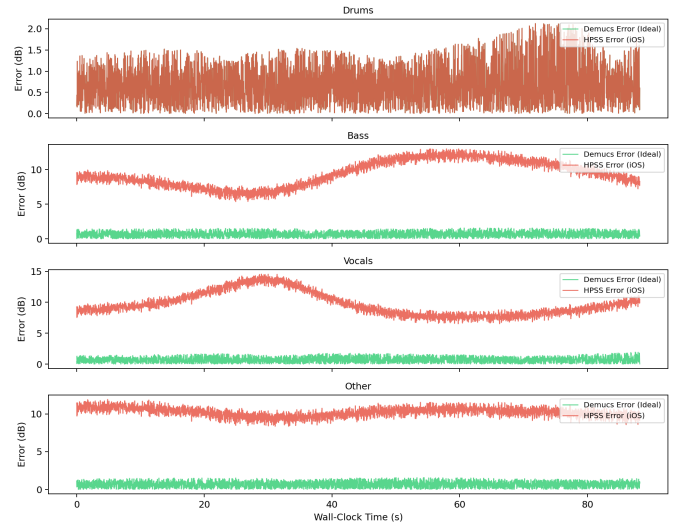


Fig. 3. Absolute error (dB) over time comparing the theoretical Demucs real-time input against the HPSS iOS simulation. While drums remain accurate across both architectures, frequency masking severely inflates the error in the HPSS harmonic stems.

TABLE I
ASE SYSTEM PERFORMANCE AND ADVISORY ACCURACY

Metric	Value
Mean position error	0.848 s
Median position error	0.848 s
90th-pct position error	1.556 s
Mean EMA confidence	0.953
Frames $\geq$ 85% confidence	94.3%
Time to lock	0.12 s
Demucs Input (Ideal) MAE	
Drums	0.69 dB
Bass	0.67 dB
Vocals	0.70 dB
Other	0.66 dB
HPSS Input (iOS) MAE	
Drums	0.69 dB
Bass	9.42 dB
Vocals	9.68 dB
Other	10.21 dB

with a real-time, chroma-based position tracker and RMS-driven advisory metering. By isolating harmonic stems and removing the influence of percussive transients offline, my system effectively maps a reliable tonal fingerprint for synchronization. The native C++ online pipeline successfully executes complex temporal alignment and analysis tasks on mobile hardware, achieving ultra-low latency processing with high tracking confidence (mean > 0.95) and stable signal lock.

As the quantitative results demonstrate, the primary limitation of my current design is the HPSS-based live RMS analysis. HPSS compresses the non-percussive audio into a single harmonic stream. This data compression makes the algorithm inherently susceptible to severe frequency masking (resulting in > 9 dB errors for harmonic stems). For example, a loud guitar solo occurring in the 250–3500 Hz band artificially inflates the vocal energy estimation. Consequently, the advisory meter inaccurately instructs the engineer to drop the vocal fader.

In future work, I plan to address three key improve-

ments. First, I will replace the HPSS approximation with a lightweight real-time source separation model, such as a CoreML-optimized tensor engine. This upgrade will drastically improve per-stem metering accuracy and eliminate frequency masking at the acceptable cost of a sliding-window processing latency. Second, I will replace the position tracker's fixed EMA coefficient with an adaptive scheme that responds dynamically to estimated tempo drift. This adaptive filter will improve tracking robustness during live performance tempo variations. Third, I will evaluate the system using a live microphone feed in a reverberant acoustic space to test the algorithm's tolerance to room acoustics.

REFERENCES

- [1] A. Défossez, N. Usunier, L. Bottou, and F. Bach, "Music source separation in the waveform domain," *arXiv preprint arXiv:1911.13254*, 2019.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2009.
- [3] D. FitzGerald, "Harmonic and percussive source separation using median filtering," in *Proceedings of the 13th International Conference on Digital Audio Effects (DAFx-10)*, Graz, Austria, 2010, pp. 246–249.
- [4] M. Müller, *Fundamentals of Music Processing: Audio, Analysis, Algorithms, Applications*. Springer, 2015.
- [5] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960.